

ICU MULTI-PATIENT ALARM PRIORITIZATION SYSTEM

A Project Report

submitted in completion of the PEP Embedded training

Submitted by

MRIDULA A (312425106119)
NANDHITHA N (312425106123)
MAHIRA SIREEN B (312425106105)
MAHALAKSHMI N (312425106104)

BACHELOR OF ENGINEERING IN ELECTRONICS AND COMMUNICATION ENGINEERING



St. JOSEPH'S INSTITUTE OF TECHNOLOGY
(An Autonomous Institution)



ANNA UNIVERSITY, CHENNAI – 600 025 SEPT 2026

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Declaration	3
2	Abstract	4
3	List of Figures	5
4	List of Tables	5
5	List of Abbreviations	5
6	CHAPTER 1 – INTRODUCTION	6
6.1	1.1 Background	6
6.2	1.2 Problem Statement	6
6.3	1.3 Objectives	6
6.4	1.4 Scope of the Project	6
6.5	1.5 Organization of the Report	6
7	CHAPTER 2 – SYSTEM ANALYSIS AND DESIGN	7
7.1	2.1 Proposed System	7
7.2	2.2 System Requirements	7
7.2.1	2.2.1 Hardware Requirements	7
7.2.2	2.2.2 Software Requirements	7
7.3	2.3 System Architecture	8
7.4	2.4 Design Diagrams	9
7.5	2.5 Pin Connections	9
8	CHAPTER 3 – IMPLEMENTATION	10
8.1	3.1 Methodology	10
8.2	3.2 Module Description	10
8.3	3.3 Demonstration Thresholds	10
8.4	3.4 Sample Code	11
9	CHAPTER 4 – RESULTS AND DISCUSSION	12
9.1	4.1 Output Screenshots	12
9.2	4.2 Testing	12
9.3	4.3 Result Analysis	13
9.4	4.4 Limitations	13
10	CHAPTER 5 – CONCLUSION AND FUTURE WORK	14
10.1	5.1 Conclusion	14
10.2	5.2 Future Enhancements	14
11	References	15

DECLARATION

We hereby declare that the project work entitled "ICU Multi-Patient Alarm Prioritization System" submitted to St. Joseph's Institute of Technology is a record of original work carried out by us under the PEP Domain.

Place: Chennai

Date: 23-09-2026

MRIDULA A (312425106119)

NANDHITHA N (312425106123)

MAHIRA SIREEN B (312425106105)

MAHALAKSHMI N (312425106104)

ABSTRACT

The ICU Multi-Patient Alarm Prioritization System is an educational embedded-systems prototype intended to demonstrate how patient pulse readings can be monitored and indicated using different alarm levels. In intensive care environments, multiple monitoring devices may generate alerts, making clear identification of abnormal readings an important design consideration. The objective of this project is to develop a microcontroller-based prototype that reads a pulse sensor, estimates heart rate, classifies the measured value into demonstration status levels, and provides visual and audible indications.

The prototype is developed using an ESP32 microcontroller, an analog three-pin pulse sensor, LEDs with current-limiting resistors, a buzzer, and a pushbutton. The sensor output is sampled by the ESP32, and the program estimates beats per minute (BPM) from detected pulse events. Based on the programmed demonstration thresholds, the LEDs indicate the status and the buzzer provides an alert for selected abnormal conditions. A pushbutton is included as a user input. The current implementation focuses on one patient channel to validate the basic sensing and alert logic before extending the design to multiple channels.

The prototype is intended for learning and demonstration only. Its thresholds and pulse-detection method are not clinically validated, and it must not be used to monitor or make decisions about real patients. Future work may include additional patient channels, improved signal processing, display integration, data logging, and testing with validated reference equipment.

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 2.1	System Architecture	8
Figure 2.2	Prototype Circuit/Wiring	9
Figure 3.1	Program Flowchart	11
Figure 4.1	Serial Monitor Output	12

LIST OF TABLES

Table No.	Title	Page No.
Table 2.2.1	Hardware Requirements	7
Table 2.2.2	Software Requirements	7
Table 2.3	ESP32 Pin Connections	8
Table 4.2.1	Functional Test Cases	12

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ICU	Intensive Care Unit
ESP32	ESP32 microcontroller family
BPM	Beats Per Minute
GPIO	General-Purpose Input/Output
ADC	Analog-to-Digital Converter

CHAPTER 1 – INTRODUCTION

1.1 Background

Patient monitoring is an important function in an Intensive Care Unit (ICU), where changes in vital signs may require attention. Embedded systems can acquire sensor signals, process measurements, and provide alerts through indicators. A multi-patient alarm prioritization concept aims to make status levels easier to distinguish when monitoring more than one patient.

1.2 Problem Statement

A basic monitoring setup may present readings without a clear visual or audible indication of which condition requires attention. This project explores a low-cost educational prototype that classifies pulse-rate estimates into programmed status categories and indicates them using LEDs and a buzzer. The current build demonstrates one patient channel; multi-patient prioritization remains an extension.

1.3 Objectives

- Interface an analog pulse sensor with the ESP32 and acquire its signal.
- Estimate BPM from detected pulse events for demonstration purposes.
- Indicate programmed status levels using LEDs and a buzzer.
- Include a pushbutton as a user-input/control element.
- Establish a single-channel prototype that can be extended to multiple patient inputs.

1.4 Scope of the Project

The scope covers a bench-top, single-patient-channel prototype using an ESP32, pulse sensor, LEDs, buzzer, and pushbutton. It demonstrates sensor acquisition, basic BPM estimation, threshold-based classification, and alert indication. It does not provide medical-grade measurements, clinical alarm prioritization, patient records, networking, or validated multi-patient monitoring.

1.5 Organization of the Report

Chapter 1 introduces the project and its objectives. Chapter 2 describes the proposed system, requirements, architecture, and connections. Chapter 3 explains the methodology and modules. Chapter 4 presents the testing approach and observed-output placeholders. Chapter 5 summarizes the prototype and future enhancements.

CHAPTER 2 – SYSTEM ANALYSIS AND DESIGN

2.1 Proposed System

The proposed prototype uses the ESP32 to read the analog pulse sensor, estimate BPM, compare the estimate with configured demonstration ranges, and drive status LEDs and an audible buzzer. A pushbutton is provided as a user input. The design is modular so that additional patient sensor channels and a central prioritization routine can be considered in future development.

2.2 System Requirements

Hardware Requirements

S.No.	Component	Role
1	ESP32 development board	Reads sensor input and controls outputs
2	Analog 3-pin pulse sensor	Provides pulse-related analog signal
3	Green LED + 220Ω resistor	Normal/demo status indication
4	Yellow LED + 220Ω resistor	Warning/demo status indication
5	Red LED + 220Ω resistor	Critical/demo status indication
6	Buzzer	Audible alert indication
7	Pushbutton	User input
8	Breadboard and jumper wires	Circuit assembly

Table 2.2.1

Software Requirements

Tool	Purpose
Arduino IDE	Writing, compiling, and uploading ESP32 firmware
ESP32 board package	Enables ESP32 support in Arduino IDE
Serial Monitor	Displays debug readings and estimated BPM

Table 2.2.2

2.3 System Architecture

Signal flow: Pulse Sensor → ESP32 ADC and pulse-detection logic → BPM estimation → Demonstration threshold classification → LED indicators and buzzer. The pushbutton provides a separate digital input to the ESP32.

S. No.	Component	ESP32 Pin	Connection / Purpose
1	Pulse Sensor (Signal)	GPIO 34	Receives analog pulse signal
2	Pulse Sensor (VCC)	3.3V	Provides power to the sensor
3	Pulse Sensor (GND)	GND	Ground connection
4	Green LED	GPIO 13	Indicates normal status
5	Yellow LED	GPIO 12	Indicates warning status
6	Red LED	GPIO 14	Indicates critical status
7	Buzzer (+)	GPIO 18	Produces an audible alarm
8	Buzzer (−)	GND	Ground connection
9	Pushbutton	GPIO 19	User input for alarm control
10	Pushbutton (Other Terminal)	GND	Used with internal pull-up configuration

Table 2.3 ESP32 Pin Connections

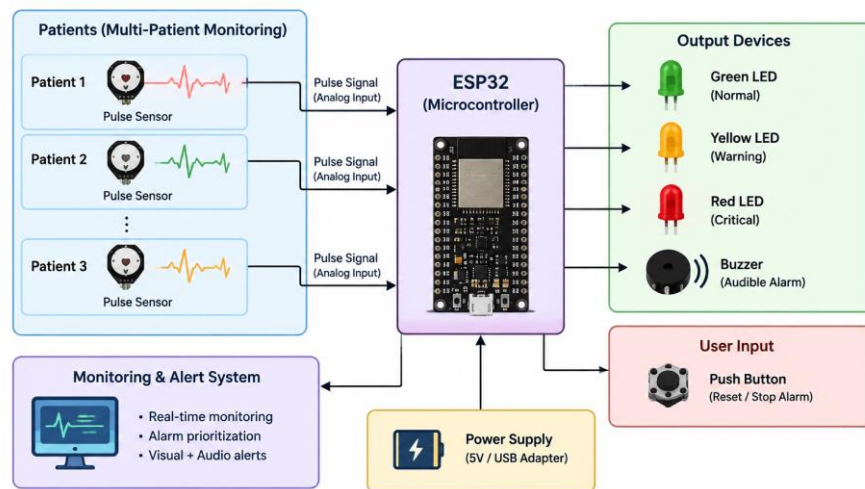


Figure 2.1 System Architecture

2.4 Design Diagrams

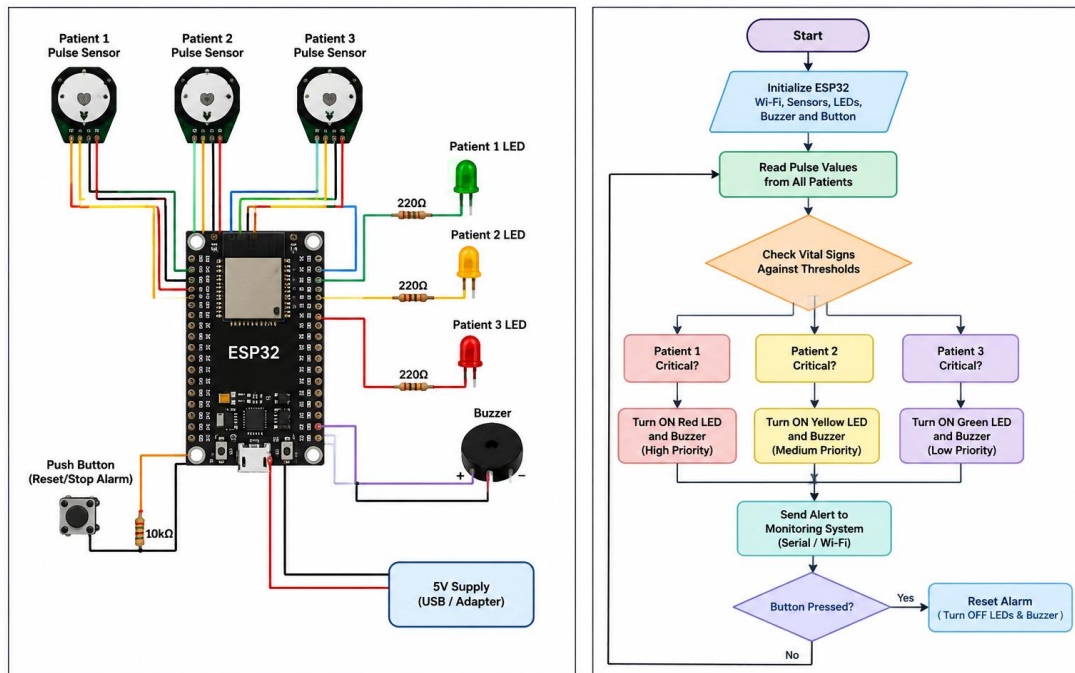


Figure 2.2 Prototype Circuit / Wiring

CHAPTER 3 – IMPLEMENTATION

3.1 Methodology

- Assemble the ESP32, pulse sensor, indicators, buzzer, and pushbutton according to the pin-connection table.
- Install ESP32 board support in Arduino IDE and select the appropriate board and port.
- Upload the firmware and open Serial Monitor to inspect sensor readings and BPM estimates.
- Detect pulse events from the analog signal and calculate the interval between detected beats.
- Convert the interval into an estimated BPM value and compare it with the programmed demonstration ranges.
- Activate the corresponding LED and buzzer output, then observe the response under different test conditions.

3.2 Module Description

Module	Function
Sensor Acquisition	Samples the analog pulse sensor using the ESP32 ADC.
Pulse Detection and BPM Estimation	Detects pulse events and estimates BPM from the time between events.
Status Classification	Maps the estimated BPM to programmed demonstration categories.
Alert Indication	Controls green/yellow/red LEDs and buzzer according to the category.
User Input and Debugging	Reads the pushbutton and prints values/status to Serial Monitor.

3.3 Demonstration Thresholds

The following ranges reflect the prototype's illustrative logic, not clinical guidance. Confirm these values against the final uploaded code before submission.

Estimated BPM	Prototype Status
Below 60	Low-rate critical/demo alert
60–100	Normal/demo
101–120	Warning/demo
Above 120	High-rate critical/demo alert

3.4 Sample Code

Insert the exact final Arduino IDE source code used in the prototype here. The code should include ADC sampling, pulse-event detection, BPM estimation, threshold classification, LED/buzzer control, pushbutton handling, and Serial Monitor output. Avoid submitting an unverified or different code version.

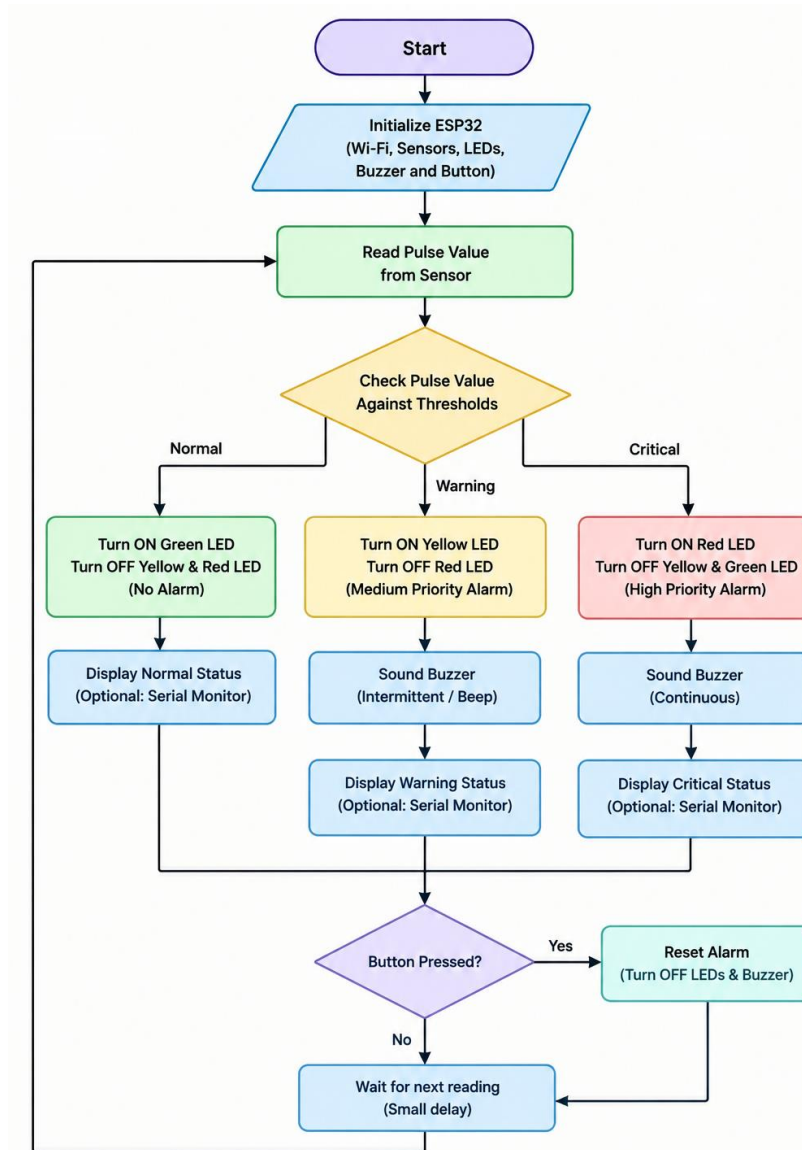
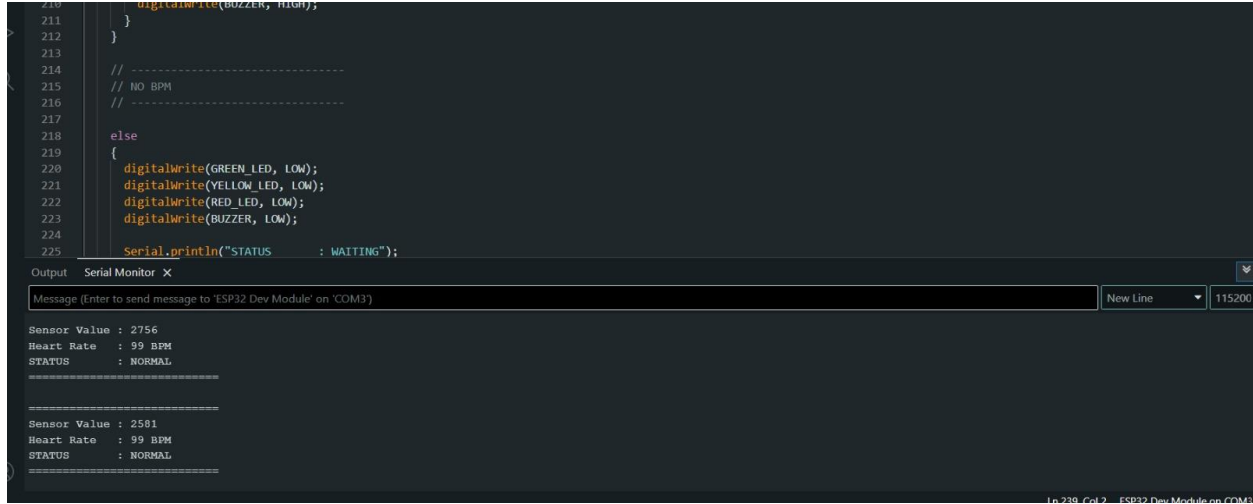


Figure 3.1 Program Flowchart

CHAPTER 4 – RESULTS AND DISCUSSION

4.1 Output Screenshots



The screenshot displays the Arduino IDE interface. The top pane shows C++ code for an ESP32, including digitalWrite calls for a buzzer and LEDs, and a Serial.println statement. The bottom pane, titled 'Serial Monitor', shows the output of the program. It displays two sets of sensor data: 'Sensor Value : 2756', 'Heart Rate : 99 BPM', and 'STATUS : NORMAL'. The output is formatted with dashed lines and a colon separator. The Serial Monitor also includes a 'New Line' button and a character count of 115200.

Figure 4.1 Serial Monitor Output

4.2 Testing

Test	Procedure	Expected/Recorded Result
Power and upload	Power the ESP32 and upload firmware	Board starts and Serial Monitor displays status; record actual result.
Sensor input	Place finger on sensor as instructed by sensor documentation	Sensor signal changes; record observed values.
BPM estimation	Observe multiple detected pulse events	BPM estimate appears after sufficient detections; record stability.
LED indication	Exercise/observe each programmed status condition safely in a test setup	Corresponding LED is activated; record actual result.
Buzzer output	Trigger a programmed alert condition	Buzzer responds according to code; record actual result.
Pushbutton	Press and release the button	Input is detected according to implemented behavior; record actual result.

Table 4.2.1

4.3 Result Analysis

The prototype is intended to demonstrate the complete path from analog sensor input to a classified visual/audible indication. Enter only results observed during testing. BPM accuracy, response time, and reliability have not been established through clinical validation. Add measured observations, limitations, and any sensor-detection issues here.

4.4 Limitations

- Only one patient channel has been assembled and tested so far.
- Pulse-sensor readings may be affected by movement, placement, ambient conditions, and signal noise.
- The BPM algorithm and threshold categories are for demonstration and are not medically validated.
- The prototype is not suitable for real-patient monitoring or clinical decisions.

CHAPTER 5 – CONCLUSION AND FUTURE WORK

5.1 Conclusion

An ESP32-based single-channel ICU alarm-indication prototype has been planned and developed to demonstrate pulse-sensor acquisition, BPM estimation, threshold-based status classification, and LED/buzzer alerts. The design provides a foundation for exploring a multi-patient alarm prioritization concept. The present implementation is an educational prototype only; it does not constitute a clinically validated monitoring system.

5.2 Future Enhancements

- Add and independently identify multiple patient sensor channels.
- Develop a central priority queue and clear alarm-state management for multiple inputs.
- Improve signal filtering, beat detection, and error handling.
- Integrate an OLED display for patient-wise BPM and status.
- Add data logging and communication features for demonstration purposes.
- Evaluate the prototype against suitable reference equipment under supervised, non-clinical testing.

REFERENCES

- [1] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems. [Online]. Available: [ESP32 Datasheet](#). [Accessed: Sep. 23, 2026].
- [2] Arduino, *Arduino IDE Documentation*. [Online]. Available: [Arduino IDE Documentation](#). [Accessed: Sep. 23, 2026].
- [3] World Famous Electronics LLC, *PulseSensor Developer Documentation*. [Online]. Available: [PulseSensor Developer Documentation](#). [Accessed: Sep. 23, 2026].
- [4] World Famous Electronics LLC, *PulseSensor Amped – Hardware and Technical Information*. [Online]. Available: [PulseSensor Amped Documentation](#). [Accessed: Sep. 23, 2026].